Notes about RL,Graph,unsupervised

目录

1 Paper Reading Improvement **3**

2 Background of Offline Reinforcement Learning(RL) **3**

2.1 Offline-RL . . . 3

2.1.1 Q-learning . . . 3

2.1.2 Ensemble-DQN . . . 3

2.1.3 Random Ensemble Mixture(REM) . . . 4

2.2 Batch-Constrained deep Q-learning(BCQ) . . . 5

2.2.1 BCQ Algorithm . . . 5

2.3 Soft Q Imitation Learning(SQIL) . . . 6

3 Application of Offline-RL in RS **7**

3.1 Problem Formulation . . . 7

3.2 Naive model . . . 7

3.3 Self-Supervised Q-learning(SQN) . . . 7

3.4 Self-Supervised Actor-Critic(SAC) . . . 8

3.5 Off-policy correction . . . 8

3.5.1 Estimating the behavior policy β . . . 8

3.5.2 Top-K Off-Policy Correction . . . 9

3.5.3 Variance Reduction Techniques . . . 9

4 Deal with Bad Training data and Noisy Labels(unsupervise) **10**

4.1 ITLM and SELF . . . 10

4.2 DivideMix . . . 10

4.2.1 Mixmatch with Label Co-refinement and Co-guessing . . . 10

4.3 Understanding and Improving Information Transfer in Multi-task Learning . . . 12

4.3.1 Model Capacity . . . 13

4.3.2 Task Covariance . . . 13

4.3.3 Optimization Scheme . . . 14

5 Application of Unsupervised Learning in RS **14**

5.1 Pretraining and Fine-tuning . . . 14

6 Graph Neural Networks Exponentially Lose power(Graph) **15**

6.1 Topological information . . . 16

6.2 Sparcity . . . 16

7 Several Ways to deal with power loss **16**
7.1 GraphSAINT . . . 16
7.1.1 Algorithm . . . 16
7.1.2 Normalization . . . 17
7.1.3 Variance . . . 17
7.1.4 Samplers . . . 18
7.2 DropEdge . . . 18
7.3 CompGCN . . . 19
7.4 InfoGraph . . . 19
7.5 Semi-supervised InfoGraph . . . 20

8 Application of Graph: Inductive Matrix Completion **20**
8.1 Enclosing subgraph extraction and Node Labeling . . . 21
8.2 Graph Neural Network Architecture . . . 21
8.3 Model Training . . . 22

9 Summary **22**
9.1 推荐系统的固有问题 . . . 22
9.2 上述方法可能存在的问题 . . . 22
9.3 可能的解决方法 . . . 22

I Paper Reading Improvement

Papers	Progress	Summary
PeterRec		code
ITLM: Iterative Trimmed Loss Minimization		mathematics
DivideMix		\
SELF: Self-Ensemble Label Filtering		Experiments
Imformation Transfer in Multi-task Learning		mathematics
Top-K Off-Policy Correction		1. Top-k Off policy correction;
An Optimistic Perspective on Offline RL		code
Off-Policy DRL without Exploration (BCQ)		mathematics, code
Self-Supervised RL for RS		code
SQIL: IL via RL with Sparse Rewards		Comparing with BC
RL to optimize Long-term User engagement in RS		Algorithm
CompGCN		code
DropEdge		mathematics, code
GNN lose expressive power		mathematics, code
GraphSaint		code
IGMC		\
InfoGraph		code

Remark 1. *1. Since some open-source code are not directly correlated with Recommendation System, I haven't read them yet. 2. The "Weakness" here means I haven't finished those things (code or mathematics proof), the progress can also be reflected in the Progressbar.*

II Background of Offline Reinforcement Learning(RL)

i. Offline-RL

1. Q-learning

Loss $\mathbb{E}_{s,a,r,s'\sim\mathcal{D}}[l_\lambda(\Delta_\theta^k(s,a,r,s'))]$.
where $\Delta_\theta^k(s,a,r,s') = Q_\theta^k(s,a) - r - \gamma \max_{a'} Q_{\theta'}^k(s',a')$ and l_λ is the Huber loss.

huber loss

$$L_\delta(a) = \begin{cases} \frac{1}{2}a^2 & for|a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta) & otherwise. \end{cases}$$

2. Ensemble-DQN

$$\text{Loss } \mathcal{L}(\theta) = \frac{1}{K}\sum_{k=1}^{K} \mathbb{E}_{s,a,r,s'\sim\mathcal{D}}[l_\lambda(\Delta_\theta^k(s,a,r,s'))],$$

where $\Delta_\theta^k(s,a,r,s') = Q_\theta^k(s,a) - r - \gamma \max_{a'} Q_{\theta'}^k(s',a')$ and l_λ is the Huber loss.

3. Random Ensemble Mixture(REM)

Loss $\mathcal{L}(\theta) = \mathbb{E}_{s,a,r,s'\sim\mathcal{D}}[E_{\alpha\sim P_\Delta}[l_\lambda(\Delta_\theta^\alpha(s,a,r,s'))]]$,
where $\Delta_\theta^\alpha = \sum_k \alpha_k Q_\theta^k(s,a) - r - \gamma \max_{a'} \sum_k \alpha_k Q_{\theta'}^k(s',a')$.

P_Δ represent a probability distribution over the standard (K-1)-simplex $\Delta^{K-1} = \{\alpha \in \mathbb{R}^K : \alpha_1 + \alpha_2 + \cdots + \alpha_K = 1, \alpha_k \geq 0, k = 1, \cdots, K\}$.

Why will REM work?

Lemma 1. *Assuming that the distribution P_Δ has full support over the entire (K-1)-simplex Δ^{K-1}, then at any global minimum of $\mathcal{L}(\theta)$, the Q-function heads Q_θ^k for $k = 1, \cdots, K$ minimize $\mathcal{L}(\alpha,\theta)$ for any $\alpha \in \Delta^{K-1}$.*

证明. Let $Q_{\alpha^*,\theta^*} = \sum_{k=1}^K \alpha_k^* Q_{\theta^*}^k(s,a)$ represents the global minima of $\mathcal{L}(\alpha,\theta)$, which can also be represented by $Q_{\alpha^*,\theta^*} = (\sum_{k=1}^K \alpha_k^* w_{\theta^*}^k) f_{\theta^*}(s,a)$, where $f_{\theta^*}(s,a) \in \mathbb{R}^D$ represent the shared features among the Q-heads. Then since $\sum_{k=1}^K \alpha_k^* w_{\theta^*}^k$ has the same dimensions as $w_{\theta^*}^k$, we can let $Q_{\theta_I}^k(s,a) = (\sum_{k=1}^K \alpha_k^* w_{\theta^*}^k) f_{\theta^*}(s,a) = Q_{\alpha^*,\theta^*}(s,a)$. Thus since all the Q-heads are the same, for all $\alpha \sim P_\Delta, \mathcal{L}(\alpha,\theta^I) = \mathcal{L}(\alpha^*,\theta^*)$, which implies that $\mathcal{L}(\theta^I) = \mathcal{L}(\alpha^*,\theta^*)$.
Therefore, if $\mathcal{L}(\theta)$ gets to the global minimum when $\theta = \theta_M$, then $\mathcal{L}(\theta^M) = \mathcal{L}(\alpha^*,\theta^*)$, which means the Q-function heads $Q_{\theta_M}^k$ for $k = 1, \cdots, K$ minimize $\mathcal{L}(\alpha,\theta)$ for any $\alpha \in \Delta^{K-1}$. □

Using this lemma, we can easily gain the following propositions:

Proposition 1. ***If*** *(a) The distribution P_Δ has full support over the entire (K-1)-simplex Δ^{K-1}, (b) Only a finite number of distince Q-functions globally minimize the REM loss.* ***Then*** *All the Q-heads represent identical Q-functions.*

证明. If the heads Q_θ^i, Q_θ^j don't converge to identical Q-values at the global minimum of $\mathcal{L}(\theta)$, then the convex combination $\alpha_i Q_\theta^i + \alpha_j Q_\theta^j$ such that $\alpha_i + \alpha_j = 1$ minimizes the loss $\mathcal{L}(\alpha,\theta)$, which means there are infinite Q-functions that globally minimize the loss and contradict the assumption above. □

Proposition 2. *Based on the above assumption (a)(b),* ***If****(c) Q^* is defined in terms of the MDP induced by the data distribution $\mathcal{D}$. (d) Q^* lies in the family of our function approximation.* ***Then*** *the common convergence point is Q^*.*

证明. The loss $\mathcal{L}(\theta)$ is lower bounded by the TD error attained by Q^*. □

Question

1. I don't really understand the constraints in this standard (K-1)-simplex problem, which according to the paper, is based on Bellman optimality constraints: $Q^*(s,a) = \mathbb{E}R(s,a) + \gamma\mathbb{E}_{s'\sim P}\max_{a'\in\mathcal{A}} Q^*(s',a')$. Also, the sentence "$\mathcal{L}(\theta)$ can be viewed as an infinite set of constraints corresponding to different mixture probality distributions" is still something strange to me.
2. The meaning of $\mathcal{L}(\theta)$ is unclear when $\mathcal{L}(\theta)$ is not the global minimum and $\mathcal{L}(\theta,\alpha)$ depend on distribution of α, which I think is the flaw of the above proof. I shall add the definition: $\mathcal{L}(\theta) = \int \mathcal{L}(\alpha,\theta)d\alpha$, which is not the only way, just to make the proof consistent.

ii. Batch-Constrained deep Q-learning(BCQ)

1. BCQ Algorithm

Purposes: To overcome **extrapolation error**(mismatch between the dataset and true state-action visitation of the current policy) in off-policy learning. BCQ is based on TD3(clipped double Q-learning). Eqn.13, target $y = r + \gamma \max_{a_i}\left[\lambda \min_{j=1,2} Q_{\theta'_j}(s',a_i) + (1-\lambda)\max_{j=1,2} Q_{\theta'_j}(s',a_i)\right]$.

Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
for $t = 1$ **to** T **do**
  Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$, $\epsilon \sim \mathcal{N}(0,\sigma)$ and observe reward r and new state s'
  Store transition tuple (s,a,r,s') in $\mathcal{B}$

  Sample mini-batch of N transitions (s,a,r,s') from $\mathcal{B}$
  $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0,\tilde{\sigma}), -c, c)$
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s',\tilde{a})$
  Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1}\sum(y - Q_{\theta_i}(s,a))^2$
  if t mod d **then**
    Update ϕ by the deterministic policy gradient:
    $\nabla_\phi J(\phi) = N^{-1}\sum \nabla_a Q_{\theta_1}(s,a)|_{a=\pi_\phi(s)}\nabla_\phi \pi_\phi(s)$
    Update target networks:
    $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i$
    $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
  end if
end for

Algorithm 1 BCQ

Input: Batch $\mathcal{B}$, horizon T, target network update rate τ, mini-batch size N, max perturbation Φ, number of sampled actions n, minimum weighting λ.
Initialize Q-networks $Q_{\theta_1}, Q_{\theta_2}$, perturbation network ξ_ϕ, and VAE $G_\omega = \{E_{\omega_1}, D_{\omega_2}\}$, with random parameters θ_1, θ_2, ϕ, ω, and target networks $Q_{\theta'_1}, Q_{\theta'_2}$, $\xi_{\phi'}$ with $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$.
for $t = 1$ **to** T **do**
  Sample mini-batch of N transitions (s,a,r,s') from $\mathcal{B}$
  $\mu,\sigma = E_{\omega_1}(s,a), \quad \tilde{a} = D_{\omega_2}(s,z), \quad z \sim \mathcal{N}(\mu,\sigma)$
  $\omega \leftarrow \text{argmin}_\omega \sum(a-\tilde{a})^2 + D_{\text{KL}}(\mathcal{N}(\mu,\sigma)||\mathcal{N}(0,1))$
  Sample n actions: $\{a_i \sim G_\omega(s')\}_{i=1}^n$
  Perturb each action: $\{a_i = a_i + \xi_\phi(s',a_i,\Phi)\}_{i=1}^n$
  Set value target y (Eqn. 13)
  $\theta \leftarrow \text{argmin}_\theta \sum(y - Q_\theta(s,a))^2$
  $\phi \leftarrow \text{argmax}_\phi \sum Q_{\theta_1}(s, a + \xi_\phi(s,a,\Phi)), a \sim G_\omega(s)$
  Update target networks: $\theta'_i \leftarrow \tau\theta + (1-\tau)\theta'_i$
  $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
end for

Several points to care about (differ from TD3):

1. Modelling the similarity between (s,a) and the state-action pairs in the batch $\mathcal{B}$ using a learned state-conditioned marginal likelihood $P^G_{\mathcal{B}}(a|s)$, which is substitued by a generative model of the batch $G_\omega(s)$, which we can sample actions from, as a reasonable approximation to $argmax_a P^G_{\mathcal{B}}(a|s)$.
2. In the target equation: $y = r + \gamma \max_{a_i}\left[\lambda \min_{j=1,2} Q_{\theta'_j}(s',a_i) + (1-\lambda)\max_{j=1,2} Q_{\theta'_j}(s',a_i)\right]$($\lambda$ should usually $> 1/2$), if we set $\lambda = 1$, this update corresponds to Clipped Double Q-learning.

Why will BCQ work?

Theorem 1. *Performing Q-learning by sampling from a batch B converges to the optimal value function under the MDP $\mathcal{M}_B$.*

ϵ_{MDP} denote the tabular extrapolation error: $\epsilon_{MDP}(s,a) = Q^\pi(s,a) - Q^\pi_{\mathcal{B}}(s,a)$. Then for any policy π, the exact form of $\epsilon_{MDP}(s,a)$ will be:

$$\begin{aligned}\epsilon_{MDP}(s,a) &= \sum_{s'}(p_M(s'|s,a) - p_{\mathcal{B}}(s'|s,a))\Big(r(s,a,s') + \gamma \sum_{a'} \pi(a'|s') Q^\pi_{\mathcal{B}}(s',a')\Big) \\ &= p_M(s'|s,a)\gamma \sum_{a'} \pi(a'|s')\epsilon_{MDP}(s',a')\end{aligned}$$

For simplicity, we denote:$\epsilon^\pi_{MDP} = \sum_s \mu_\pi(s) \sum_a \pi(a|s)|\epsilon_{MDP}(s,a)|$.

Lemma 2. *For all reward functions, $\epsilon^\pi_{MDP} = 0$ if and only if $p_{\mathcal{B}}(s'|s,a) = p_M(s'|s,a)$ for all $s' \in S$ and (s,a) such that $\mu_\pi(s) > 0$ and $\pi(a|s) > 0$.*

Theorem 2. *For a deterministic MDP and all reward functions, $\epsilon^\pi_{MDP} = 0$ if and only if the policy π is batch-constrained. Furthermore, if $\mathcal{B}$ is coherent, then such a policy must exist if the start state $s_0 \in \mathcal{B}$.*

Theorem 3. *Given the Robbins-Monro stochastic convergence conditions on the lr α, and standard sampling requirements from the environment, BCQL converges to the optimal value function Q^*.*

Theorem 4. *Given a deterministic MDP and coherent batch B, along with the Robbins-Monro stochastic convergence conditions on the learning rate α and standard sampling requirements on the batch B, BCQL converges to $Q^\pi_{\mathcal{B}}(s,a)$ where $\pi^*(s) = argmax_{a s.t. (s;a)\in B} Q^\pi_{\mathcal{B}}(s,a)$ is the optimal batch-constrained policy.*

iii. Soft Q Imitation Learning(SQIL)

SQIL is summarized in the following algorithm, where Q_θ is the soft Q function, $\mathcal{D}_{demo}$ are demonstrations, δ^2 is the squared soft Bellman error:

$$\delta^2(\mathcal{D}, r) \triangleq \frac{1}{|\mathcal{D}|} \sum_{(s,a,s')\in\mathcal{D}} \Big(Q_\theta(s,a) - \Big(r + \gamma \log\Big(\sum_{a'\in\mathcal{A}} \exp(Q_\theta(s',a'))\Big)\Big)\Big)^2 \tag{1}$$

Algorithm 1 Soft Q Imitation Learning (SQIL)

```
Require λ_samp ∈ ℝ_{≥0}, D_demo
Initialize D_samp ← ∅
while Q_θ not converged do
    θ ← θ − η∇_θ(δ²(D_demo, 1) + λ_samp δ²(D_samp, 0)) {See Equation 1}
    Sample transition (s, a, s') with imitation policy π(a|s) ∝ exp(Q_θ(s, a))
    D_samp ← D_samp ∪ {(s, a, s')}
end while
```

As the imitation policy learns to behave like the expert, a growing number of expert-like transitions get

added to the buffer $\mathcal{D}_{samp}$ with an assigned reward of 0.

III Application of Offline-RL in RS

i. Problem Formulation

The next item recommendation problem can be formulated as an MDP, which can be defined by tuples consisting of $(\mathcal{S}, \mathcal{A}, \mathbf{P}, R, \rho_0, \gamma)$, and $\mathbf{s} = G(x_{1:t})$. RL seeks a target policy $\pi_\theta(a|\mathbf{s})$ to maximize:$\max_{\pi_\theta} \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$, $\quad R(\tau) = \sum_{t=0}^{|\tau|} \gamma^t r(\mathbf{s}_t, a_t)$. The gradient of this cumulative rewards can be derived by: $\mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)\nabla_\theta \log \pi_\theta(\tau)]$.

In online RL, the gradient: $\sum_{\tau\sim\pi_\theta} R(\tau)\nabla_\theta \log \pi_\theta(\tau) \approx \sum_{\tau\sim\pi_\theta} \Big[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \pi_\theta(a_t|s_t)\Big]$.

ii. Naive model

1. The user-item interation sequence can be represented as $x_{1:t} = \{x_1, x_2, \cdots, x_{t-1}, x_t\}$. Thus the recommendation can be seen as a classification problem with CE-loss:

$$\mathbf{y}_{t+1} = f(\mathbf{s}_t) = f(G(x_{1:t})) \tag{2}$$

2. We can also model the state transition $\mathbf{P} : \mathcal{S}\times\mathcal{A}\times\mathcal{S}$ with a recurrent neural network: $s_{t+1} = f(s_t, \mathbf{u}_{a_t})$. Then the policy $\pi_\theta(a|s)$ is then modeled with a simple softmax:$\pi_\theta(a|s)$ is then modeled with a simple softmax:$\pi_\theta(a|\mathbf{s}) = \dfrac{exp(\mathbf{s}^T\mathbf{v}_a/T)}{\sum_{a'\in\mathcal{A}} exp(\mathbf{s}^T\mathbf{v}_{a'}/T)}$.

iii. Self-Supervised Q-learning(SQN)

Because of the distribution discrepancy between current policy π and the policy in batch β(which is the same as extrapolation error), we can not use the equation $Q(s_t, a_t) = r(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a')$(some actions may not happen in the logged data). However, due to the greedy selection of $Q(s_{t+1}, \cdot)$, the estimation of $Q(s_t, a_t)$ itself is unbiased. Thus we have two losses here:

$$\begin{aligned}
L_s &= -\sum_{i=1}^{n} Y_i \log p_i, where p_i = \frac{e^{y_i}}{\sum_{i'=1}^{n} e^{y_{i'}}} \\
L_q &= (r(s_t, a_t) + \gamma \max_{a'} Q(\mathbf{s}_{t+1}, a') - Q'(\mathbf{s}_t, a_t))^2, \quad Q(\mathbf{s}_t, a_t) = \delta(\mathbf{s}_t \mathbf{h}_t^T + b) \\
L_{SQN} &= L_s + L_q
\end{aligned}$$

After creating G', Q' as copies of G and Q, respectively, we will randomly set Q and Q' as the net and the target net, to substitude Q and Q' in the above equation, Then update all parameters.

remark

(1) Q-learning is utilized as a regularizer here; (2) Keeping the self-supervised layer as the source of recommendations can help avoid any off-policy correction issues. (3) The lack of negative rewards

(since the data are collected by experts, which corresponds to the senario in imitation learning) in Q-learning does not affect the methods since the RL output layer is trained on positive actions. (4)CE-loss will help the model to choose the action in the batch.

iv. Self-Supervised Actor-Critic(SAC)

"actor": the self-supervised head; "critic": the Q-learning head. Then we can use the Q-values as weights for the self-supervised loss: $L_A = L_s \cdot Q(\mathbf{s}_t, a_t)$. Then $L_{SAC} = L_A + L_s$.

remark

The action in AC is based on policy gradient which is on-policy, so to enhance stability, the essay stop the gradient flow and fix the Q-values when they are used in that case. Then train the actor and critic jointly.

v. Off-policy correction

Question

All the equations in the subsection has no proof in the original paper, yet I cannot finish the proof by myself, I will try again when I have more time.

Through importance weighting, we can address distribution mismatch (or data biases), Consider trajectory $\tau = (s_0, a_0, s_1, \cdots)$ sampled according to a behavior policy β, the off-policy-corrected gradient estimator is then:

$$\sum_{\tau\sim\beta} \frac{\pi_\theta(\tau)}{\beta(\tau)} \Big[\sum_{t=1}^{|\tau|} R_t \nabla_\theta \log(\pi_\theta(a_t|s_t)) \Big] \tag{3}$$

where $\dfrac{\pi_\theta(\tau)}{\beta(\tau)} = \dfrac{\rho(s_0)\prod_{t=0}^{|\tau|} P(s_{t+1}|s_t, a_t)\pi(a_t|s_t)}{\rho(s_0)\prod_{t=0}^{|\tau|} P(s_{t+1}|s_t, a_t)\beta(a_t|s_t)} = \prod_{t=0}^{|\tau|} \dfrac{\pi(a_t|s_t)}{\beta(a_t|s_t)}$. However, this approximation has high variance. To reduce the variance, we can first ignoring the terms after time t in the chained products, and further taking a first-order approximation:

$$\prod_{t'=0}^{|\tau|} \frac{\pi(a_{t'}|s_{t'})}{\beta(a_{t'}|s_{t'})} \approx \prod_{t'=0}^{t} \frac{\pi(a_{t'}|s_{t'})}{\beta(a_{t'}|s_{t'})} = \frac{P_{\pi_\theta}(s_t)}{P_\beta(s_t)} \frac{\pi(a_t|s_t)}{\beta(a_t|s_t)} \approx \frac{\pi(a_t|s_t)}{\beta(a_t|s_t)}.$$

Finally, we have the biased estimator of the policy gradient with lower variance:

Off-Policy Correction

$$\sum_{\tau\sim\pi_\theta} R(\tau)\nabla_\theta \log \pi_\theta(\tau) \approx \sum_{\tau\sim\beta} \Big[\sum_{t=0}^{|\tau|} \frac{\pi_\theta(a_t|s_t)}{\beta(a_t|s_t)} R_t \nabla_\theta \log \pi_\theta(a_t|s_t) \Big]$$

1. Estimating the behavior policy β

Using the equations above, we need the policy β. There are several ways to deal with this:

1. (Naive method)Estimate the behavior policy β independent of user state by aggregate action frequency throughout the corpus.
2. (Depend on Neural Network)Adopt a context-dependent neural estimator. Then For each state-action pair (s,a) collected, we estimate the probability $\hat{\beta}_{\theta'}(a|s)$ parameterised by θ'.

Differences between the update of β and π

(1) While the main policy π_θ is effectively trained using a weighted softmax to take into account of long term reward, the behavior policy head $\beta_{\theta'}$ is trained using only the state-action pairs; (2) While the main policy head π_θ is trained using only items on the trajectory with non-zero reward, the behavior policy $\beta_{\theta'}$ is trained using all of the items on the trajectory to avoid introducing bias in the β estimate.

2. Top-K Off-Policy Correction

Assumption: (1) The expected reward of a set of non-repetitive items equal to the sum of the expected reward of each item in the set. (2) $\Pi_\Theta(A'|s) = \prod_{a \in A'} \pi_\theta(a|s)$. Note that set A' contain duplicate items, which are removed to form a non-repetitive set A. Then we can change π_θ in the policy gradient equation to α_θ: $\sum_{\tau \sim \pi_\theta} \Big[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log \alpha_\theta(a_t|s_t) \Big]$, where $\alpha_\theta = 1 - (1 - \pi_\theta(a|s))^K$ is the probabiliy that an item a appears in the final non-repetitive set A. Here $K = |A'| > A = k$.

Finally, we get the top-K off-policy correction factor:

$$\sum_{\tau \sim \beta} \frac{\alpha_\theta(\tau)}{\beta(\tau)} \Big[\sum_{t=0}^{|\tau|} R_t \nabla_\theta \log(\alpha_\theta(a_t|s_t)) \Big] = \sum_{\tau \sim \beta} \Big[\sum_{t=0}^{|\tau|} \frac{\pi_\theta(a_t|s_t)}{\beta(a_t|s_t)} \frac{\partial \alpha(a_t|s_t)}{\partial \pi(a_t|s_t)} R_t \nabla_\theta \log(\pi_\theta(a_t|s_t)) \Big] \tag{4}$$

The top-K policy adds an additional multiplier of: $\lambda_K(s_t, a_t) = \dfrac{\partial \alpha(a_t|s_t)}{\partial \pi(a_t|s_t)} = K(1 - \pi_\theta(a_t|s_t))^{K-1}$.

3. Variance Reduction Techniques

In spite of the first-order approximation to reduce variance in the gradient estimate, it still suffers from large variance due to large importance weight of $\omega(s,a) = \frac{\pi(a|s)}{\beta(a|s)}$. Then here comes several techniques:

1. **Weight Capping** Simply cap the weight as: $\overline{\omega}_c(s,a) = \min\left(\frac{\pi(a|s)}{\beta(a|s)}, c\right)$.
2. **Normalized Importance Sampling(NIS)** Weight normalization: $\overline{\omega}_n(s,a) = \dfrac{\omega(s,a)}{\sum_{(s',a') \sim \beta} \omega(s',a')}$.
3. **Trusted Region Policy Optimization(TRPO)**. Prevents the new policy π from deviating from the behavior policy by adding a regularization that penalized the KL divergence of these two policies.

IV Deal with Bad Training data and Noisy Labels(unsupervise)

i. ITLM and SELF

Motivation: (1) In the general training settings, the evolution of training accuracy (as a function of training epochs) is different for clean and bad samples. **ITLM** will help recover the ground truth (with linear convergence rate) in generalized linear models with standard statistical assumptions. (2) DNNs have been shown to over-fit a dataset when being trained with noisy labels for a long enough time. Then this method **SELF**(self-ensemble label filtering) will progressively filter out the wrong labels during training.

Algorithm 1: ITLM or SELF

Input: Samples $\{s_i\}_{i=1}^n$.
(SELF) ensembling momentum $\beta, 0 < \beta < 1$.
(ITLM) fraction of samples α, number of rounds T.
Initialize: $\theta_0 \leftarrow arg\min_\theta \sum_{i\in[n]} f_\theta(s_i)$; (SELF) $\overline{z}_i \leftarrow 0$, $M_{best} = M_0$.
(SELF): **while** $acc(M_{t+1}, S_{val}) \geq acc(M_t, S_{val})$ **do**
  $S_t \leftarrow \{(x_i, y_i) \in S | \overline{z}_i \leftarrow M_{best}(x_i); \overline{z}_i \leftarrow \beta\overline{z}_{i-1} + (1-\beta)\overline{z}_i, y = argmax(\overline{z}_i)\}$.
  $\theta_{t+1} = ModelUpdate(\theta_t, S_t, t)$.
end
(ITLM): **for** $t = 1$ **to** T **do**
  $S_t \leftarrow arg\min_{S:|S|=\lfloor \alpha n \rfloor} \sum_{i\in S} f_{\theta_t}(s_i)$
  $\theta_{t+1} = ModelUpdate(\theta_t, S_t, t)$.
end

ii. DivideMix

First, fit a two-component GMM to the loss l using the EM-algorithm. For each sample, its clean probability $w_i = p(g|l_i), i = 1, \cdots, B$(the posterior probability), where g is the Gaussian component with smaller mean (smaller loss).

Confidence Penalty for Asymmetric Noise

To forbid the model to produce over-confident (low entropy) predictions, which leads to most samples having near-zero normalized loss, then the GMM cannot effectively distinguish clean and noisy samples based on the loss distribution. To address this issue, the confidence penalty is introduced:

$$\mathcal{H} = -\sum_c p^c_{model}(x;\theta)\log(p^c_{model}(x;\theta)) \tag{5}$$

1. Mixmatch with Label Co-refinement and Co-guessing

The following two tricks are added to the method MixMatch:

1. **Label co-refinement**: Linearly combine the ground-truth label y_b with the network's prediction p_b (averaged across multiple augmentations of x_b), guided by the clean probability w_b produced by *the other network*: $\overline{y}_b = w_b y_b + (1 - w_b)p_b$. Then perform a sharpening function on the refined

Algorithm 1: DivideMix. Line 4-8: co-divide; Line 17-18: label co-refinement; Line 20: label co-guessing.

1 **Input:** $\theta^{(1)}$ and $\theta^{(2)}$, training dataset $(\mathcal{X}, \mathcal{Y})$, clean probability threshold τ, number of augmentations M, sharpening temperature T, unsupervised loss weight λ_u, Beta distribution parameter α for MixMatch.
2 $\theta^{(1)}, \theta^{(2)} = \text{WarmUp}(\mathcal{X}, \mathcal{Y}, \theta^{(1)}, \theta^{(2)})$ *// standard training (with confidence penalty)*
3 **while** $e < \text{MaxEpoch}$ **do**
4 $\quad \mathcal{W}^{(2)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(1)})$ *// model per-sample loss with $\theta^{(1)}$ to obtain clean proabability for $\theta^{(2)}$*
5 $\quad \mathcal{W}^{(1)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(2)})$ *// model per-sample loss with $\theta^{(2)}$ to obtain clean proabability for $\theta^{(1)}$*
6 $\quad$ **for** $k = 1, 2$ **do** *// train the two networks one by one*
7 $\quad\quad \mathcal{X}_e^{(k)} = \{(x_i, y_i, w_i) | w_i \geq \tau, \forall (x_i, y_i, w_i) \in (\mathcal{X}, \mathcal{Y}, \mathcal{W}^{(k)})\}$ *// labeled training set for $\theta^{(k)}$*
8 $\quad\quad \mathcal{U}_e^{(k)} = \{x_i | w_i < \tau, \forall (x_i, w_i) \in (\mathcal{X}, \mathcal{W}^{(k)})\}$ *// unlabeled training set for $\theta^{(k)}$*
9 $\quad\quad$ **for** iter $= 1$ **to** num_iters **do**
10 $\quad\quad\quad$ From $\mathcal{X}_e^{(k)}$, draw a mini-batch $\{(x_b, y_b, w_b); b \in (1, ..., B)\}$
11 $\quad\quad\quad$ From $\mathcal{U}_e^{(k)}$, draw a mini-batch $\{u_b; b \in (1, ..., B)\}$
12 $\quad\quad\quad$ **for** $b = 1$ **to** B **do**
13 $\quad\quad\quad\quad$ **for** $m = 1$ **to** M **do**
14 $\quad\quad\quad\quad\quad \hat{x}_{b,m} = \text{Augment}(x_b)$ *// apply m^{th} round of augmentation to x_b*
15 $\quad\quad\quad\quad\quad \hat{u}_{b,m} = \text{Augment}(u_b)$ *// apply m^{th} round of augmentation to u_b*
16 $\quad\quad\quad\quad$ **end**
17 $\quad\quad\quad\quad p_b = \frac{1}{M} \sum_m \text{p}_{\text{model}}(\hat{x}_{b,m}; \theta^{(k)})$ *// average the predictions across augmentations of x_b*
18 $\quad\quad\quad\quad \bar{y}_b = w_b y_b + (1 - w_b) p_b$
// refine ground-truth label guided by the clean probability produced by the other network
19 $\quad\quad\quad\quad \hat{y}_b = \text{Sharpen}(\bar{y}_b, T)$ *// apply temperature sharpening to the refined label*
20 $\quad\quad\quad\quad \bar{q}_b = \frac{1}{2M} \sum_m \left(\text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(1)}) + \text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(2)})\right)$
// co-guessing: average the predictions from both networks across augmentations of u_b
21 $\quad\quad\quad\quad q_b = \text{Sharpen}(\bar{q}_b, T)$ *// apply temperature sharpening to the guessed label*
22 $\quad\quad\quad$ **end**
23 $\quad\quad\quad \hat{\mathcal{X}} = \{(\hat{x}_{b,m}, \hat{y}_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented labeled mini-batch*
24 $\quad\quad\quad \hat{\mathcal{U}} = \{(\hat{u}_{b,m}, q_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented unlabeled mini-batch*
25 $\quad\quad\quad \mathcal{L}_{\mathcal{X}}, \mathcal{L}_{\mathcal{U}} = \text{MixMatch}(\hat{\mathcal{X}}, \hat{\mathcal{U}})$ *// apply MixMatch*
26 $\quad\quad\quad \mathcal{L} = \mathcal{L}_{\mathcal{X}} + \lambda_u \mathcal{L}_{\mathcal{U}} + \lambda_r \mathcal{L}_{\text{reg}}$ *// total loss*
27 $\quad\quad\quad \theta^{(k)} = \text{SGD}(\mathcal{L}, \theta^{(k)})$ *// update model parameters*
28 $\quad\quad$ **end**
29 $\quad$ **end**
30 **end**

$$\text{label: } \hat{y}_b = Sharpen(\bar{y}_b, T) = \bar{y}_b^{c\frac{1}{T}} \Big/ \sum_{c=1}^{C} \bar{y}_b^{c\frac{1}{T}}, \text{ for } c = 1, 2, \cdots, C.$$

2. **co-guessing**: Use the ensemble of predictions from both networks to "co-guess" the labels for unlabeled samples, which can produce more reliable guessed labels.

MixMatch: Combine $\hat{\mathcal{X}}$ and $\hat{\mathcal{U}}$ to get $\mathcal{W}$: $\mathcal{W} = Shuffle(Concat(\hat{\mathcal{X}}, \hat{\mathcal{U}}))$. Then we can apply MixUp to labeled data and entries from $\mathcal{W}$:

$$\mathcal{X}' = MixUp(\hat{\mathcal{X}}_i, \mathcal{W}_i); i \in (1, \cdots, |\hat{\mathcal{X}}|); \quad \mathcal{U}' = MixUp(\hat{\mathcal{U}}_i, \mathcal{W}_{i+|\hat{\mathcal{X}}|}); i \in (1, \cdots, |\hat{\mathcal{U}}|)$$

One thing to take care is that $\hat{\mathcal{X}}$, $\hat{\mathcal{U}}$ are the mini-batch chosen from the training dataset. Thus MixUp is performed every step in the training epoch.

Then the loss on $\mathcal{X}'$ is the CE-loss and the loss on $\mathcal{U}'$ is the MSE loss:

$$\mathcal{L}_x = -\frac{1}{|\mathcal{X}'|} \sum_{x,p \in \mathcal{X}'} \sum_c p_c \log(p^c_{model}(x;\theta)) \tag{6}$$

$$\mathcal{L}_u = \frac{1}{|\mathcal{U}'|} \sum_{x,q \in \mathcal{U}'} ||q - p_{model}(x;\theta)||_2^2 \tag{7}$$

Under high levels of noise, the network would be encouraged to predict the same class to minimize the loss. However, to prevent assigning all samples to a single class, a regularization term is introduced:

$$\mathcal{L}_{reg} = \sum_c \pi_c \log\left(\pi_c \Big/ \frac{1}{|\mathcal{X}'| + |\mathcal{U}'|} \sum_{x \in \mathcal{X}' + \mathcal{U}'} p^c_{model}(x;\theta)\right) \tag{8}$$

Finally, the total loss is: $\mathcal{L} = \mathcal{L}_\mathcal{X} + \lambda_u \mathcal{L}_\mathcal{U} + \lambda_r \mathcal{L}_{reg}$.

MixUp

***mixup**: construct virtual training examples (the original paper does not have the second equation):*

$$\begin{aligned}
&\lambda \sim Beta(\alpha, \alpha) \\
&\lambda' = \max(\lambda, 1-\lambda) \\
&\tilde{x} = \lambda x_i + (1-\lambda) x_j, \quad \textit{where } x_i, x_j \textit{ are raw input vectors} \\
&\tilde{y} = \lambda y_i + (1-\lambda) y_j, \quad \textit{where } y_i, y_j \textit{ are one-hot label encodings}
\end{aligned}$$

where (x_i, y_i) and (x_j, y_j) are two examples drawn at random from our training data, and $\lambda \in [0,1]$.

iii. Understanding and Improving Information Transfer in Multi-task Learning

Several notions: k: the number of tasks; m_i: the number of data samples of task i.; $X_i \in \mathbb{R}^{m_i \times d}$: the covariates of task i; $y_i \in R^{m_i}$: the labels of task i. $B \in \mathbb{R}^{d \times r}$: shared module in MTL model; $A_i \in \mathbb{R}^r$: seperate output module for task i.

Positive vs. negative transfer: If training through equation (9) improves over just training the target task alone, then we say the source task transfers *positively* to the target task. *Negative* transfer is the converse of positive transfer.

Column span: For a matrix X, its column span is the set of all linear combinations of the column vectors of X.

This paper has several valuable points:

1. (**Optimization Scheme**)We can turn to multitask and small down the weight of this task to deal with the noisy labels in this task;
2. (**Task Covariance**)With the alignment matrices, we can simply apply covariance alignments to these tasks, which will decrease the covariance between different models, thus make sure the transfer between these models are positive;
3. (**Model Capacity**)The output dimension of the shared module can not be too big, or there will be no transfer between these tasks at all.

The objective of finding an MTL model is to minimize the following equation:

$$f(A_1, A_2, \cdots, A_k; B) = \sum_{i=1}^{k} \alpha_i L(g(X_i B) A_i, y_i) \tag{9}$$

1. Model Capacity

Proposition 3. *If $r \geq k$, then there exists an optimum B^* and $\{A_i^*\}_{i=1}^k$ of the following equation:*

$$f(A_1, A_2, \cdots, A_k; B) = \sum_{i=1}^{k} ||X_i B A_i - y_i||_F^2 \tag{10}$$

which means limiting the shared module's capacity is necessary to enforce information transfer. If the shared module is too small, then tasks may interfere negatively with each other.

2. Task Covariance

To improve multi-task training, we can add an alinment matrix R_i before its input X_i passes through the shared module B. Then the total loss becomes:

$$\hat{f}(A_1, \cdots, A_k; R_1, \cdots, R_k) = \sum_{i=1}^{k} L(g(Z_i B) A_i, y_i) = \sum_{i=1}^{k} L(g(X_i R_i B) A_i, y_i) \tag{11}$$

Quantify how many data points is needed from the source to guarantee positive transfer:

Theorem 5. *For $i = 1, 2$, $(X_i, y_i) \in (\mathbb{R}^{m_i \times d}, \mathbb{R}^{m_i})$: two linear regressiong tasks with parameters $\theta_i \in \mathbb{R}^d$; m_i number of samples in task i. Suppose that each row of the source task X_1 is drawn independently from a distribution with covariance $\sum_1 \in \mathbb{R}^{d \times d}$ and bounded l_2-norm $\sqrt{L}$. Assume that $\theta_1^T \sum_1 \theta_1 = 1$ w.l.o.g.*
Let $c \in [\kappa(X_2)\sin(\theta_1, \theta_2), 1/3]$ denote the desired error margin. Denote by (B^*, A_1^*, A_2^*), the optimal MTL solution. With probability $1 - \delta$ over the randomness of (X_1, y_1), when m_1 is at least on the order of $(\kappa^2(\sum_1) \cdot \kappa^4(X_2) \cdot ||y_2||^2)/c^4$, concretely, when

$$m_1 \geq \max\Big(\frac{L||\sum_1||\log\frac{d}{\delta}}{\lambda_{min}^2(\sum_1)}, \frac{\kappa(\sum_1)\kappa^2(X_2)}{c^2}||y_2||^2, \frac{\kappa^2(\sum_1)\kappa^4(X_2)}{c^4}\sigma_1^2 \log\frac{1}{\delta}\Big)$$

we have that $||B^* A_2^* - \theta_2||/||\theta_2|| \leq 6c + \frac{1}{1-3c}\frac{||\varepsilon_2||}{||X_2\theta_2||}$.
Bisides, this paper also propose a metrix called covariance similarity score to measure the similarity between two tasks. Given $X_1 \in \mathbb{R}^{m_1 \times d}$ and $X_2 \in \mathbb{R}^{m_2 \times d}$, we can measure the similarity in three steps: : (a) The covariance matrix is $X_1^T X_1$. (b) Find the best rank-r_1 approximation to be $U_{1,r_2} D_{1,r_1} U_{1,r_1}^T$, where r1 is chosen to contain 99% of the singular values. (c) Apply step (a),(b) to X_2, compute the score:

$$\textit{Covariance similarity score} := \frac{||(U_{1,r} D_{1,r_1}^{1/2})^T U_{2,r_2} D_{2,r_2}^{1/2}||_F}{||U_{1,r} D_{1,r_1}^{1/2}||_F \cdot ||U_{2,r_2} D_{2,r_2}^{1/2}||_F} \tag{12}$$

3. Optimization Scheme

We can add a re-weighting scheme in the presence of label noise. The full procedure and the theoratical assurance are shown below:

Algorithm 2: An SVD-based task re-weighting scheme

Input: k tasks: $(X_i, y_i) \in (\mathbb{R}^{m\times b}, R^m)$; a rank parameter $r \in \{1, 2, \cdots, k\}$

Output: A weight vector: $\{\alpha_1, \alpha_2, \cdots, \alpha_k\}$

Let $\theta_i = X^T y_i$

$U_r, D_r, V_r = SVD_r(\theta_1, \theta_2, \cdots, \theta_k)$, i.e. the best rank-$r$ approximation to the θ_i's.

Let $\alpha_i = ||\theta_i^T U_r||$, for $i = 1, 2, \cdots, k$.

Proposition 4. *Let the shared module have capacity $r \leq k$. Given k tasks with the same covariates $X \in \mathbb{R}^{m\times b}$ but different labels $\{y_i\}_{i=1}^k$. Let X be full rank and UDV^T be its SVD. Let $Q_r Q_r^T$ be the best rank-r approximation to $\sum_{i=1}^k \alpha_i U^T y_i y_i^T U$. Let $B^* \in \mathbb{R}^{d\times r}$ be an optimal solution for the re-weighted loss. Then the column span of B^* is equal to the column span of $(X^T X)^{-1} V D Q_r$.*

V Application of Unsupervised Learning in RS

Basic notations: $\mathcal{S}$: source domain (with a large number of use interactions); $\mathcal{T}$: target domain (user labels are usually very limited); $\mathcal{U}$: the set of users shared in both domains. $\mathbf{x}^u = \{x_1^u, \cdots, x_n^u\}(x_i^u \in \mathcal{X}), i.e.(u, x^u) \in S$: the unsupervised interaction sequence. $\mathcal{X}$: the set of items in $\mathcal{S}$; $(u, y) \in \mathcal{T}$: the supervised label $y \in \mathcal{Y}$ for the instance with userID u in $\mathcal{T}$.

i. Pretraining and Fine-tuning

The first method is the unsupervised autoregressive approach. And the joint probability $p(\mathbf{x}^u; \Theta)$ of each user sequence is represented by the product of the conditional distributions over the items:

$$p(\mathbf{x}^u; \Theta) = \prod_{i=1}^{n} p(x_i^u | x_1^u, \cdots, x_{i-1}^u; \Theta) \tag{13}$$

which is so much like the pretraining strategy in gpt-2.

Then as for the fine-tuning process, we can perform a series of causal CNN operations on the embedding of $\mathbf{x}^u = \{x_1^i, \cdots, x_n^u, [TCL]\}$, we obtain the last hidden layer matrix. Then just place a linear classification layer on top of the final hidden vector: $\mathbf{o} = \mathbf{h}_n \mathbf{W} + \mathbf{b}$. After a softmax and argmax, we get the recommended item.

The second method takes account of two-side contexts. Randomly mask a certain percentage of items of $\mathbf{x}^u$ by filling in the mask symbols in the sequence. (Just like Bert do, the difference is that here uses a dialated non-causal CNN, which I think can be easily replaced by Transformer Encoder). Denote the masked sequence as $\overline{\mathbf{x}}_\Delta^u$, $\mathbf{x}_\Delta^u = \{x_{\Delta_1}^u, \cdots, x_{\Delta_m}^u\}(1 < m < t)$ be the masked interactions, then the

probability of $p(\mathbf{x}^u_\Delta)$ is given as:

$$p(\mathbf{x}^u_\Delta;\Theta) = \prod_{i=1}^{m} p(x^u_{\Delta_i}|\hat{x}^u;\Theta) \tag{14}$$

Which is so much like the strategy in Bert pretraining.

Then as for the fine-tuning process, we can simply add [TCL]s at the start and the end positions of $\mathbf{x}^u$, i.e., $\mathbf{x}^u = \{[TCL], \mathbf{x}^u_1, \cdots, \mathbf{x}^u_n, [TCL]\}$, and accordingly, $\mathbf{o} = (\mathbf{h}_0 + \mathbf{h}_n)\mathbf{W} + b$ or $\mathbf{o} = (\sum_{i=1}^n h_i)\mathbf{W} + \mathbf{b}$.

fine-tuning objective

BPR loss for top-N item recommendation task: $R_{BPR}(\mathcal{T};\Theta;v;\theta) = -\sum_{(u,y)\in\mathcal{T}} \log\delta(\mathbf{o}_y - \mathbf{o}_{y-})$;

CE loss for the user profile classification tasks: $R_{CE}(\mathcal{T};\Theta;v;\theta) = -\sum_{(u,y)\in\mathcal{T}} \log p(y|u)$.

where $y-$ is a false label randomly sampled from $\mathcal{Y}\backslash y$. Besides, PeterRec only updates θ and v (including $\mathbf{W}$&$\mathbf{b}$) by freezing pre-trained parameters Θ.

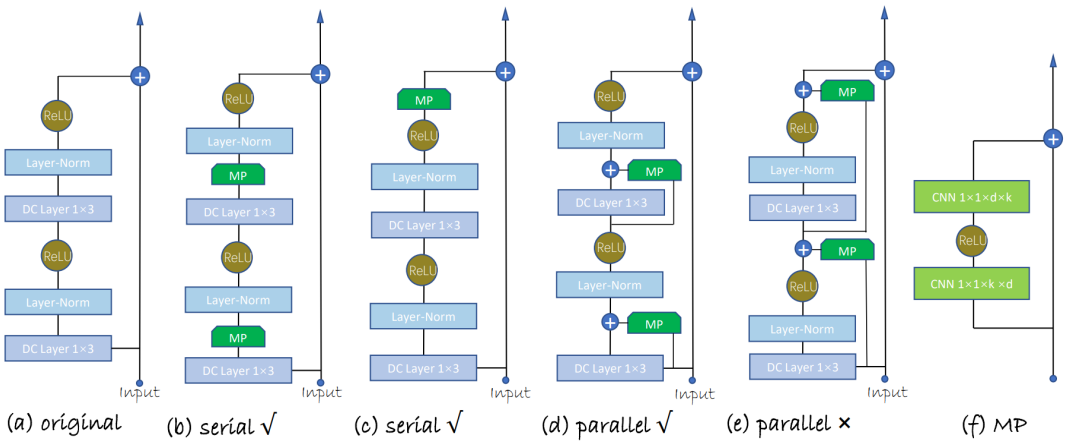


Figure 3: Model patch (MP) and insertion methods. (a) is the original pre-trained residual block; (b) (c) (d) (e) are the fine-tuned residual blocks with inserted MPs; and (f) is the MP block. + is the addition operation. 1×3 is the kernel size of dilated convolutional layer.

The **Model Patch Structure** proposed in the paper is $\mathcal{H}_{MP}(\tilde{\mathbf{E}}) = \tilde{\mathbf{E}} + \phi_{up}(\sigma(\phi_{down}(\tilde{\mathbf{E}})))$, which corresponds to θ and v in the above fine-tuning objective. The several ways to inject the structure in the original model are described in the above figure.

VI Graph Neural Networks Exponentially Lose power(Graph)

GCN on Undirected Graphs

Propagation rule of a layer is defined as: $\mathbf{h}_v^{(l+1)} = \sigma\Big((\mathbf{W}^{(l)})^T \sum_{u\in\mathcal{V}} \tilde{\mathcal{A}}_{v,u}\mathbf{h}_u^{(l)}\Big)$ or $\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathcal{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$.

GCN on Multi-Relatinoal Graphs

$\mathbf{H}^{k+1} = \sigma(\hat{\mathbf{A}}\mathbf{H}^k\mathbf{W}^k_r)$ or $\mathbf{h}_r = \sigma(\sum_{(u,r)\in\mathcal{N}(v)} \mathbf{W}_r\mathbf{h}_u)$, where $\mathbf{W}^k_r$ denotes the relation specific parameters of the model.

i. Topological information

Define the augmented normalized Laplacian of G by $\tilde{\Delta} := I_N - \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ and set $P := \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$.

Proposition 5. *Let $\lambda_1 \leq \cdots \leq \lambda_N$ be the eigenvalue of P. Then we have $-1 < \lambda_1, \lambda_{N-M} < 1$ and $\lambda_{N-M+1} = \cdots = \lambda_N = 1$. In particular, we have $\lambda := \max_{n=1,\cdots,N-M} |\lambda_n| < 1$. Furthur, $e_m := \tilde{D}^{1/2}u_m$ for $m \in [M]$ are the basis of the eigen space associated with the eigenvalue 1.*

Theorem 6. *For any initial value $X^{(0)}$, the output of $l-th$ layer $X^{(l)}$ satisfieds $d_{\mathcal{M}}(X^{(l)}) \leq (s\lambda)^l d_{\mathcal{M}}(X^{(0)})$. In particular, $\delta_{\mathcal{M}}(X^{(l)})$ exponentially converges to 0 when $s\lambda < 1$.*

Specifically, for any $X \in \mathcal{M}$, if two nodes $i, j \in V$ are in a same connected component and their degrees are identical, then, the column vectors of X that correspond to nodes i and j are identical. In higher layers, the features cannot help us distinguish these two nodes.
This is the "information loss" of GCNs in the limit of infinite layers.

the rate $s\lambda$ in the above theorem depends on the **spectra of the augmented normalized Laplacian**, which is determined by the topology of the underlying graph G. Hence, this result explicitly relates the topological information of graphs and asymptotic behaviors of graph NNs.

ii. Sparcity

Theorem 7. *Consider a GCN on the Erdos-Renyi graph $G_{N,p}$ such that $\frac{\log N}{Np} = o(1)$ as a function of N. For any $\epsilon > 0$, if the supremum s of the maximum singular values of weights in the GCN satisfies $s < s_0 := \frac{1}{7}\sqrt{\frac{Np-p+1}{\log(4N/\epsilon)}}$, then, for sufficiently large N, the GCN satisfies the condition of **Theorem 6** with probability at least $1 - \epsilon$.*

Under this condition, the upper bound $s_0 \to \infty$ as $N \to \infty$. It means that if the graph is sufficiently large and not extremely sparse, most GCNs suffer from the information loss, though $s > 1$.

VII Several Ways to deal with power loss

i. GraphSAINT

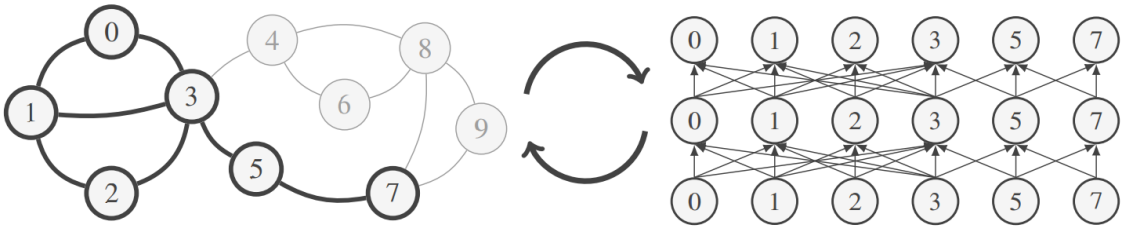
According to the discussion in 6.2, randomly sampling edges in an underlying graph, which will make the graph sparser, can deal with the over-smoothing phenomena.

over-smoothing

The over-smoothing phenomenon implies that the node features will converge to a fixed point as the network depth increases, which is right the case discussed in the previous section.

1. Algorithm

1. Extract appropriately connected subgraphs so that little information is lost when propagating within the subgraphs; 2. Combine information of many subgraphs together so that the training process overall learns good representation of the full graph.



$\mathcal{G}_s = \texttt{SAMPLE}(\mathcal{G})$ Full GCN on $\mathcal{G}_s$

2. Normalization

If v is sampled(i.e. $v \in \mathcal{V}_s$), we can compute the aggregated feature of v as:

$$\zeta_v^{(l+1)} = \sum_{u\in\mathcal{V}} \frac{\tilde{A}_{v,u}}{\alpha_{u,v}} (\mathbf{W}^{(l)})^T \mathbf{x}_u^{(l)} \mathbb{I}_{u|v} = \sum_{u\in\mathcal{V}} \frac{\tilde{A}_{v,u}}{\alpha_{u,v}} \tilde{\mathbf{x}}_u^{(l)} \mathbb{I}_{u|v}; \quad \mathbb{I}_{u|v} = \begin{cases} 1 & \text{if } (u,v) \in \mathcal{E}_s \\ 0 & \text{if } v \in \mathcal{V}_s \wedge (u,v) \notin \mathcal{E}_s \end{cases}$$

Where does $\alpha_{u,v}$ come from?

Proposition 6. *$\zeta_v^{(l+1)}$ is an unbiased estimator of the aggregation of v in the full $(l+1)^{th}$ GCN layer, if $\alpha_{u,v} = \frac{p_{u,v}}{p_v}$. i.e., $\mathbb{E}(\zeta_v^{(l+1)}) = \sum_{u\in\mathcal{V}} \tilde{\mathbf{A}}_{v,u} \tilde{\mathbf{x}}_u^{(l)}$, where $p_{u,v} = p_{v,u}$ is the probability of an edge $(u,v) \in \mathcal{E}_s$ being sampled in a subgraph, and p_v is the probability of a node $v \in \mathcal{V}$ being sampled.*

证明. *When $v \in \mathcal{G}_s$, $\mathbb{E}(\zeta_v^{(l+1)}) = \mathbb{E}(\sum_{u\in\mathcal{V}} \frac{\tilde{A}_{v,u}}{\alpha_{u,v}} \tilde{\mathbf{x}}_u^{(l)} \mathbb{I}_{u|v}) = \sum_{u\in\mathcal{V}} \frac{\tilde{A}_{u,v}}{\alpha_{u,v}} \tilde{x}_u^{(l)} \mathbb{E}(\mathbb{I}_{u|v}) = \sum_{u\in\mathcal{V}} \frac{\tilde{A}_{u,v}}{\alpha_{u,v}} \tilde{x}_u^{(l)} \frac{p_{u,v}}{p_v}$.*

Furthur, let L_v be the loss on v in the output layer. The minibatch loss is caculated as $L_{batch} = \sum_{v\in\mathcal{G}_s} L_v/\lambda_v$, where $\lambda_v = |\mathcal{V}| \cdot p_v$ so that: $\mathbb{E}(L_{batch}) = \frac{1}{|\mathbb{G}|} \sum_{\mathcal{G}_s\in\mathbb{G}} \sum_{v\in\mathcal{V}_s} \frac{L_v}{\lambda_v} = \frac{1}{|\mathcal{V}|} \sum_{v\in\mathcal{V}} L_v$.

3. Variance

Let e be the edge connecting u,v, and $\mathbf{b}_e^{(l)} = \tilde{A}_{v,u} \tilde{x}_u^{(l-1)} + \tilde{A}_{u,v} \tilde{x}_v^{(l-1)}$. Define:

$$\zeta = \sum_l \sum_{v\in\mathcal{G}_s} \frac{\zeta_v^{(l)}}{p_v} = \sum_l \sum_{v,u} \frac{\tilde{A}_{v,u}}{p_v \alpha_{u,v}} \mathbb{I}_v \mathbb{I}_{u|v} = \sum_l \sum_e \frac{\mathbf{b}_e^{(l)}}{p_e} \mathbf{I}_e^{(l)}$$

Where does p_e come from?

Theorem 8. *Then under independent edge sampling with budget m $(\sum p_e = m)$, the optimal edge progabilities to minimize the sum of variance of each ζ's dimension is given by: $p_e = \frac{m}{\sum_{e'} ||\sum_l \mathbf{b}_{e'}^{(l)}||} ||\sum_l \mathbf{b}_e^{(l)}||$.*

proof. *First assume that the* $\mathbf{b}_e^{(l)}$ *is a scalar and denote it by* $b_e^{(l)}$. *Then:*

$$\begin{aligned} Var(\zeta) &= \sum_{e,l} \Big(\frac{b_e^{(l)}}{p_e}\Big)^2 Var(\mathbb{I}_e^{(l)}) + 2 \sum_{e,l_1<l_2} \frac{b_e^{(l_1)} b_e^{(l_2)}}{p_e^2} Cov(\mathbb{I}_e^{(l_1)}, \mathbb{I}_e^{(l_2)}) \\ &= \sum_{e,l} \frac{(b_e^{(l)})^2}{p_e} - \sum_{e,l} (b_e^{(l)})^2 + 2 \sum_{e,l_1<l_2} \frac{b_e^{(l_1)} b_e^{(l_2)}}{p_e^2} (p_e - p_e^2) \\ &= \sum_{e} \frac{(\sum_l b_e^{(l)})^2}{p_e} - \sum_{e} \Big(\sum_l b_e^{(l)}\Big)^2 \end{aligned}$$

Since $m = \sum_e p_e$, $\sum_e \frac{(\sum_l b_e^{(l)})^2}{p_e} m = \sum_e \Big(\frac{\sum_l b_e^{(l)}}{\sqrt{p_e}}\Big)^2 \sum_e (\sqrt{p_e})^2 \geq (\sum_{e,l} b_e^{(l)})^2$ *(Cauchy inequality), The equality is achieved when* $\Big|\frac{\sum_l b_e^{(l)}}{\sqrt{p_e}}\Big| \propto \sqrt{p_e} \Longrightarrow p_e \propto \Big|\sum_l b_e^{(l)}\Big| \Longrightarrow p_e = \frac{m}{\sum_{e'} |\sum_l \mathbf{b}_{e'}^{(l)}|} \Big|\sum_l \mathbf{b}_e^{(l)}\Big|$.

4. Samplers

1. **Random node sampler**: Sample $|\mathcal{V}_s|$ nodes from $\mathcal{V}$ randomly, according to a node probability distribution $P(u) \approx ||\tilde{\mathbf{A}}_{:,u}||^2$.
2. **Random edge sampler** Calculating $\mathbf{b}_e^{(l)}$ reauires computing $\tilde{\mathbf{x}}_v^{(l-1)}$, which is complex. Therefore, ignore $\tilde{\mathbf{x}}_v^{(l)}$ to make the edge probability dependent on the graph topology only. Therefore, choose $p_e \propto \tilde{\mathbf{A}}_{v,u} + \tilde{\mathbf{A}}_{u,v} = \frac{1}{deg(u)} + \frac{1}{degv}$.
3. **Random walk based samplers**: When ignoring activations, L layers can be represented as a single layer with edge weights given by $\mathbf{B} = \tilde{A}^L$. If it were possible to pick pairs of nodes(whether or not they are directly connected in the original $\tilde{A}$) independently, then set $p_{u,v} \propto \mathbf{B}_{u,v} + \mathbf{B}_{v,u}$, where $\mathbf{B}_{u,v}$ can be interpreted as the probability of a random walk to start at u and end at v in L hops (and $\mathbf{B}_{u,v}$ vice-versa).

ii. DropEdge

At each training epoch, the DropEdge technique drops out a certain rate of edges of the input graph by random. Vp non-zero elements of the adjacency matrix $\mathbf{A}$ will be enforced to be zeros, where V is the total number of edges. $\mathbf{A}_{drop} = \mathbf{A} - \mathbf{A}'$, then after a re-normalization trick on $\mathbf{A}_{drop}$, leading to $\hat{\mathbf{A}}_{drop}$, $\hat{\mathbf{A}}$ will be replaced by $\tilde{\mathbf{A}_{drop}}$ for propagation and training.

Why can DropEdge deal with over-smoothing?

1. *(subspace)* $\mathcal{M} := \{\mathbf{EC}|\mathbf{C} \in \mathbb{R}\}$**:** *subspace of* $\mathbb{R}^{N\times C}$, *where* $\mathbf{E}^T\mathbf{E} = \mathbf{I}_M, M \leq N$.
2. *(*$\epsilon -$ *smoothing) Given a GCN, if* $d_{\mathcal{M}}(\mathbf{H}^{(l)}) < \epsilon, \forall l \geq L$, *then we call the* ϵ*-smoothing of node features happens for a GCN, where* $\mathbb{M}$ *is independent to the input features.*
3. *(the* $\epsilon -$ *smoothing layer)* $l^*(\mathcal{M}, \epsilon) := min_l\{d_{\mathcal{M}}(\mathbf{H}^{(l)}) < \epsilon\}$.
4. *(the relaxed* $\epsilon -$ *smoothing layer):* $\hat{l}(\mathcal{M}, \epsilon) = \lceil \frac{\log(\epsilon/d_{\mathcal{M}}(\mathbf{X}))}{\log s\lambda} \rceil$, s *is the supremum of the filters' singular values over all layers, and* λ *is the second largest eigenvalue of* $\hat{\mathbf{A}}$. *Besides,* $\hat{l} \geq l^{*4}$.

Theorem 9. *$\mathcal{G}$: the original graph; $\mathcal{G}'$: the graph after dropping certain edges. Given a small value of ϵ, we assume $\mathcal{G}$ and $\mathcal{G}'$ will encounter the ϵ-smoothing issue with regard to subspaces $\mathcal{M}$ and $\mathcal{M}'$, respectively. Then, eigher of the following inequalities holds after suffient edges removed.*

1. *The relaxed smoothing layer only increases: $\hat{l}(\mathcal{M}, \epsilon) \leq \hat{l}(\mathcal{M}', \epsilon)$;*
2. *The information loss is decreased: $N - dim(\mathcal{M}) > N - dim(\mathcal{M}')$.*

iii. CompGCN

Since using only node-level embeddings will lose power as the layer gets to infinity, it's a great idea to also incorporate the relational features into the embeddings and update processes.
Comparing to GCN update equation $\mathbf{h}_r = \sigma\Big(\sum_{(u,r)\in\mathcal{N}(v)} \mathbf{W}_r \mathbf{h}_u\Big)$, the update equation of CompGCN is:

$$\mathbf{h}_v^{(k+1)} = \sigma\Big(\sum_{(u,v)\in\mathcal{N}(v)} \mathbf{W}_{\lambda(r)}^k \phi(\mathbf{h}_u^k, \mathbf{h}_r^k)\Big); \quad \mathbf{h}_r^{k+1} = \mathbf{W}_{rel}^k \mathbf{h}_r^k \tag{15}$$

where $\mathbf{W}_{\lambda(r)} \in \mathbb{R}^{d_1 \times d_0}$ is the relation-type specific parameter and $\mathbf{W}_{rel} \in \mathbb{R}^{d_1 \times d_0}$ is a learnalbe transformation matrix which projects all the relations to the same embedding space as nodes and allows them to be utilized in the next GompGCN layer. In CompGCN, we use direction specfic weights, i.e., $\lambda(r) = dir(r)$, given as:

$$\mathbf{W}_{dir(r)} = \begin{cases} \mathbf{W}_O, & r \in \mathcal{R} \\ \mathbf{W}_I, & r \in \mathcal{R}_{inv} \\ \mathbf{W}_S, & r = \top(\text{self-loop}) \end{cases} \tag{16}$$

Scaling with Increasing Number of Relations

Instead of independently defining an embedding for each relation, they are expressed as a linear combination of a set of basis vectors. Finally, let $\{\mathbf{v}_1, \mathbf{v}_2, \cdots, \mathbf{v}_\mathcal{B}\}$ be a set of learnable basis vectors. Then initial relation representation is given as: $\mathbf{z}_r = \sum_{b=1}^{\mathcal{B}} \alpha_{br} \mathbf{v}_b$, where $\alpha_{br} \in \mathbb{R}$ is relation and basis specific learnable scalar weight.

iv. InfoGraph

Formally, the k-th layer of a GNN is:(patch representations)

$$h_v^{(k)} = COMBINE^{(k)}\Big(h_v^{(k-1)}, AGGREGATE^{(K)}\Big(\Big\{\Big(h_v^{(k-1)}, h_u^{(k-1)}, e_{uv}\Big) : u \in \mathcal{N}(v)\Big\}\Big)\Big) \tag{17}$$

Then summarize feature vectors at all depths of the graph neural network into a single feature vector:

$$h_\phi^i = CONCAT(\{h_i^{(k)}\}_{k=1}^K), \quad H_\phi(G) = READOUT(\{h_\phi^i\}_{i=1}^N) \tag{18}$$

where h_ϕ^i is the summarized patch representation centered at node i and $H_\phi(G)$ is the global representation after applying READOUT. Then the object of InfoGraph is given as:

$$\hat{\phi}, \hat{\psi} = arg \max_{\phi,\psi} \sum_{G \in \mathcal{G}} \frac{1}{|G|} \sum_{u \in G} I_{\phi,\psi}(\mathbf{h}_\phi^u; H_\phi(G)) \tag{19}$$

$$I_{\phi,\psi}(h_\phi^i(G); H_\phi(G)) := \mathbb{E}_{\mathbb{P}}[-sp(-T_{\phi,\psi}(h_\phi^i(x), H_\phi(x)))] - \mathbb{E}_{\mathbb{P}\times\tilde{\mathbb{P}}}[sp(T_{\phi,\psi}(h_\phi^i(x'), H_\phi(x)))] \tag{20}$$

where sp means softplus.

v. Semi-supervised InfoGraph

Based on the previous unsupervised model, try to minimize the following objective function:

$$L_{total} = \sum_{i=1}^{|\mathbb{G}^L|} L_{supervised}(y_\phi(G_i), o_i) + \lambda \sum_{j=1}^{|\mathbb{G}^L|+|\mathbb{G}^U|} L_{unsupervised}(h_\phi(G_j); H_\phi(G_j)) \tag{21}$$

However, supervised tasks and unsupervised tasks may favor different information of a different semantic space, simply combineing the two loss functions using the same encoder may lead to "negative transfer". So, deploy two encoder models: **supervised encoder** and **unsupervised encoder**. Then define a loss term that encourages the representations learned by the two encoder to have high mutual information, at all levels of representations. Then the total loss function can be defined by:

$$L_{total} = \sum_{i=1}^{|\mathbb{G}^L|} L_x(y_\phi(G_i), o_i) + \sum_{j=1}^{|\mathbb{G}^L|+|\mathbb{G}^U|} L_u(h_\varphi(G_j); H_\varphi(G_j)) - \lambda \sum_{j=1}^{|\mathbb{G}^L|+|\mathbb{G}^U|} \frac{1}{|G_i|} \sum_{k=1}^{K} I(H_\phi^k(G_j); H_\varphi^k(G_j)) \tag{22}$$

Note: (1) this formulation can be seen as a special instance of the **student-teacher** framework. However, instead of enforcing the predictions of the student model to be similar to the teacher model, this paper enforce the transfer of knowledge from the teacher model to the student model via mutual-information maximization at various levels of representations. (2) In order to reduce the computational cost, we can choose one layer randomly from the encoder to do mutual-information maximization.

VIII Application of Graph: Inductive Matrix Completion

Flaws in matrix factorization

Factorization technique means decomposing a rating r_{ij} into $\mathbf{w}_i^T \mathbf{h}_j$, the inner product of user i' s and item j's latent feature vectors $\mathbf{w}_i$ and $\mathbf{h}_j$, respectively. However, when the rating matrix has changed values or has new rows/columns added, it ofter requires a complete retraining to get the new embeddings.

Flaws in IMC(Inductive Matrix Completion)

In IMC, rating is decomposed by $r_{ij} = \mathbf{x}_i^T \mathbf{Q} \mathbf{y}_j$. But to accurately predict missing entries, IMC methods have strong constraints on the content quality, which often leads to inferior performance when high-quality content is not available. Moreover, in some extreme settings, there is not any content available for user, such as a website where users are completely anonymous. In these cases, inductive matrix completion seems impossible.

i. Enclosing subgraph extraction and Node Labeling

Algorithm 1 ENCLOSING SUBGRAPH EXTRACTION

input: h, target user-item pair (u, v), the bipartite graph G
output: the h-hop enclosing subgraph $G^h_{u,v}$ for (u, v)
$U = U_{fringe} = \{u\}, V = V_{fringe} = \{v\}$
for $i = 1, 2, \ldots, h$ **do**
  $U'_{fringe} = \{u_i : u_i \sim V_{fringe}\} \setminus U$
  $V'_{fringe} = \{v_i : v_i \sim U_{fringe}\} \setminus V$
  $U_{fringe} = U'_{fringe},\ V_{fringe} = V'_{fringe}$
  $U = U \cup U_{fringe},\ V = V \cup V_{fringe}$
  Let $G^h_{u,v}$ be the vertex-induced subgraph from G using vertices U, V
  Remove edge (u, v) from $G^h_{u,v}$.
end for
return $G^h_{u,v}$

Note: $\{u_i : u_i \sim V_{fringe}\}$ is the set of nodes that are adjacent to at least one node in V_{fringe} with any edge type.

Node Labeling

If a user-type node is included at the ith hop, we will give it a label $2i$, if an item-type node is included at the ith, we will give it $2i+1$. Then the one-hot encoding of these node labels will be treated as the initial node features x_0. This can be easily incorporated in the above algorithm.

ii. Graph Neural Network Architecture

Instead of extracting node embeddings in the previous methods, IGMC will applies a graph-level GNN to the enclosing subgraph around (u, v) and map the subgraph to the rating.
The whole process can be given as follow (R-GCN is adopted here):

$$\mathbf{x}_i^{l+1} = \mathbf{W}_o^l \mathbf{x}_i^l + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \mathbf{W}_r^l x_j^l \tag{23}$$

$$\mathbf{h}_i = concat(\mathbf{x}_i^1, \mathbf{x}_i^2, \cdots, \mathbf{x}_i^L) \tag{24}$$

$$\mathbf{g} = concat(\mathbf{h}_u, \mathbf{h}_v) \tag{25}$$

$$\hat{r} = \mathbf{w}^T \sigma(\mathbf{W}\mathbf{g}) \tag{26}$$

iii. Model Training

Loss function is chosen to be MSE error:$\mathcal{L} = \frac{1}{|\{(u,v)|\Omega_{u,v}=1\}|} \sum_{(u,v):\Omega_{u,v}=1} (R_{u,v} - \hat{R}_{u,v})^2$.

Adjacent rating regularization: To deal with the drawback that this GCN will fail to take the magnitude of ratings into consideration, the ARR regularizer is proposed:

$$\mathcal{L}_{ARR} = \sum_{i=1,2,\cdots,|\mathcal{R}|-1} ||\mathbf{W}_{r_{i+1}} - \mathbf{W}_{r_i}||_F^2 \tag{27}$$

Final loss is given by: $\mathcal{L}_{final} = \mathcal{L} + \lambda \mathcal{L}_{ARR}$.

IX Summary

i. 推荐系统的固有问题

1. 被推荐的候选物品空间可能会非常大.
2. 用户的品味和喜好可能会随着时间非常多变.
3. 相对于大量的动作空间, 数据会显得十分稀疏.
4. 随着时间增长, 动作空间可能仍然会随之增大.
5. 推荐系统无法通过自己与环境交互去获得训练数据，只能利用以往的数据进行训练.(offline)

ii. 上述方法可能存在的问题

1. 在 SQN(3.3) 和 SAC(3.4) 两个方法中, 都用到了对现所处 state 的 embedding, 而对于物品, 则直接使用物品数量作为 Q-function 的输出维度, 因此有着很大的改进空间; 目前已有的很多 RL 方法, 同样有着对物品的 embedding. 但即便如此, RL 的这几个问题仍然很明显: (1) 状态空间太大, 行动空间也太大, 因此计算 $Q(s,a)$ 通常有着数据量不够的问题 (模型维数太大, 参数量太大); (2) offline-RL 的一些问题, 已经可以被解决的相对比较不错; (3) 对现处状态的 embedding 还有优化空间.
2. 在无监督迁移学习的方法 (5.1) 中, 显然没有考虑到这是一个 MDP, 并无时间序列特征, 无法对用户的喜好变化建模.
3. GNN 做 Matrix Completion(8) 可以对不同的 user 和 item 进行打分, 但是在推荐系统中, 显然无法做到一一去计算, 无法对每一个 item 都去抽取一些子图, 去打分, 时间代价太大, 因此这种方法当作其它方法的辅助即可.

iii. 可能的解决方法

1. 借鉴 Bert 或 GPT-2 pretraining-finetuning 的方式, 可以对现在的状态得到更好的 state embedding. 用于强化学习中 (GPT-2, 是应该可以捕捉到时间序列信息的; 虽然在论文中, non-causal CNN 的效果不如 causal CNN, 但是用到 transformer decoder, 结果可能会有所不同; 用 Bert 对全局进行 embedding 也有着独到的好处).
2. (1) 当拿到一个新的 user, 以往的 interaction 的信息 (当该信息很少时, 才是 GNN 发挥显著作用的地方), 利用 IGMC 中的二分图的方式, 从这个 user 出发, 找到一个二分图, 将该二分图投入 GNN,

得到一个 state 的 embedding. 而随着 user 的 interaction 的增加, 只需要扩充该二分图即可; (2) 为了体现时间序列性, 可以改进 Node Labeling 的方法, 将到现在的时间距离考虑到 Labeling 中. (显然, 提取全局信息, 便可以用到前面所提到的 CompGCN 以及 InfoGraph)

3. 对 item 进行 embedding 的方式及其使用方法已找到一些相关工作, 正在调研中 ·····

1. 两种方法：(1) 给定一个 state 和一个 action, 设计一个 Q-network, 计算 Q(s,a), 利用这里的 Q 值放入强化学习方法中进行迭代更新; (2) 给定一个 state, 经过一个网络, 最后一层会得到 N 个输出, 作为 N 个 action 的 Q-value; 第 (1) 种方法中, 训练过程中, 只需要给定 state 和 action 即可, 计算会很快, 但是在真正做推荐的时候, 最朴素的方法是将所有的 item 都与这里的 state 一起输入一遍 Q-network, 计算量便会很大; 第 (2) 种方法中, 需要一个大模型记住所有的 item, 才可以只从 state 得到所有的 Q-value,

2. 已有的 simulated user 的训练方式, 似乎并没有用 RL 去训练一个 simulated user.